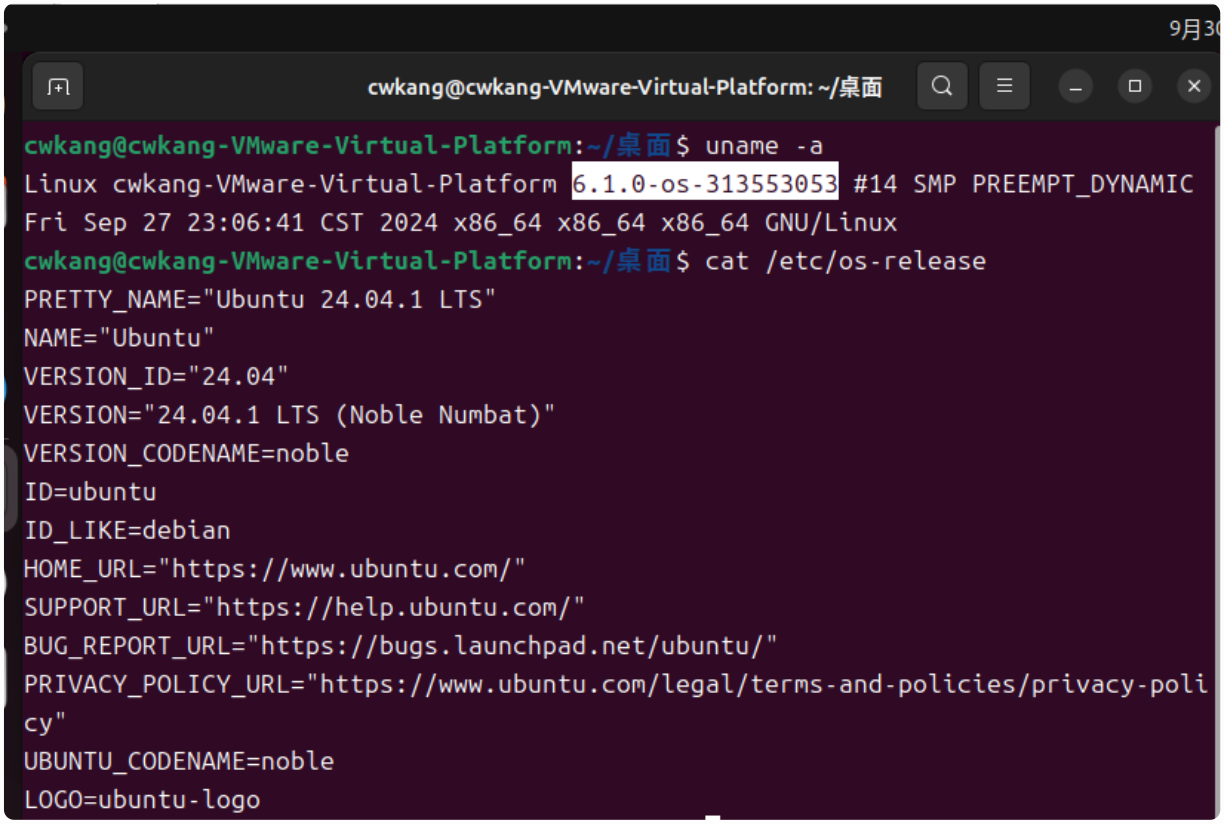
# Operating System - Assignment 1

## I. Compiling the Linux Kernel

### i. Answer screenshot

Execute `uname -a` and `cat /etc/os-release`




### ii. Kernel compilation process

#### Step 1. Install Required Packages

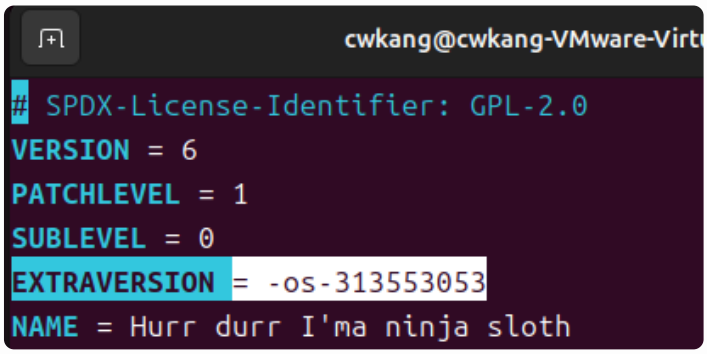
```
$ sudo apt-get update
$ sudo apt-get install build-essential libncurses-dev bison flex libssl-dev libel
```

#### Step 2. Kernel Configuration

```
$ cp -v /boot/config-$(uname -r) .config
$ make menuconfig
```

#### Step 3. Add the suffix and Remove the suffix of "+"

```
$ vim Makefile
$ vim scripts/setlocalversion
```




```
cwkang@cwkang-VMware-Virtual-Platform:~/桌面/linux$ git show 9d193b126e1701d6dd91c17f643258d596d2bddf
commit 9d193b126e1701d6dd91c17f643258d596d2bddf
Author: cwkang <wei020789@gmail.com>
Date:   Thu Sep 26 00:09:54 2024 +0800

    not diplay the suffix +

diff --git a/scripts/setlocalversion b/scripts/setlocalversion
index af4754a35..afcc64d9e 100755
--- a/scripts/setlocalversion
+++ b/scripts/setlocalversion
@@ -139,7 +139,8 @@ elif [ "${LOCALVERSION+set}" != "set" ]; then
        # If the variable LOCALVERSION is set (including being set
        # to an empty string), we don't want to append a plus sign.
        scm=$(scm_version --short)
-       res="$res${scm:++}"
+       # res="$res${scm:++}"
+       res="$res${scm:+}"
 fi

 echo "$res"
```

### Step 4. Modify TRUSTED_KEYS and REVOCATION_KEYS

```
$ vim .conifg
```

- before:

```
# Certificates for signature checking
#
CONFIG_MODULE_SIG_KEY="certs/signing_key.pem"
CONFIG_MODULE_SIG_KEY_TYPE_RSA=y
# CONFIG_MODULE_SIG_KEY_TYPE_ECDSA is not set
CONFIG_SYSTEM_TRUSTED_KEYRING=y
CONFIG_SYSTEM_TRUSTED_KEYS="debian/canonical-certs.pem"
CONFIG_SYSTEM_EXTRA_CERTIFICATE=y
CONFIG_SYSTEM_EXTRA_CERTIFICATE_SIZE=4096
CONFIG_SECONDARY_TRUSTED_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_HASH_LIST=""
CONFIG_SYSTEM_REVOCATION_LIST=y
CONFIG_SYSTEM_REVOCATION_KEYS="debian/canonical-revoked-certs.pem"
# CONFIG_SYSTEM_BLACKLIST_AUTH_UPDATE is not set
# end of Certificates for signature checking
```

- after:

```
# Certificates for signature checking
#
CONFIG_MODULE_SIG_KEY="certs/signing_key.pem"
CONFIG_MODULE_SIG_KEY_TYPE_RSA=y
# CONFIG_MODULE_SIG_KEY_TYPE_ECDSA is not set
CONFIG_SYSTEM_TRUSTED_KEYRING=y
CONFIG_SYSTEM_TRUSTED_KEYS=""
CONFIG_SYSTEM_EXTRA_CERTIFICATE=y
CONFIG_SYSTEM_EXTRA_CERTIFICATE_SIZE=4096
CONFIG_SECONDARY_TRUSTED_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_KEYRING=y
CONFIG_SYSTEM_BLACKLIST_HASH_LIST=""
CONFIG_SYSTEM_REVOCATION_LIST=y
```

**Step 5. Compile the Kernel**

Use the number of cores on machine to speed up the process

```
cwkang@cwkang-VMware-Virtual-Platform:~/桌面/linux$ echo $(nproc)
4
```

```
$ make -j4
```

#### Step 6. Install the Kernel Modules

```
$ sudo make modules_install
```

#### Step 7. Install the Kernel

```
$ sudo make install
```

#### Step 8. Update Bootloader

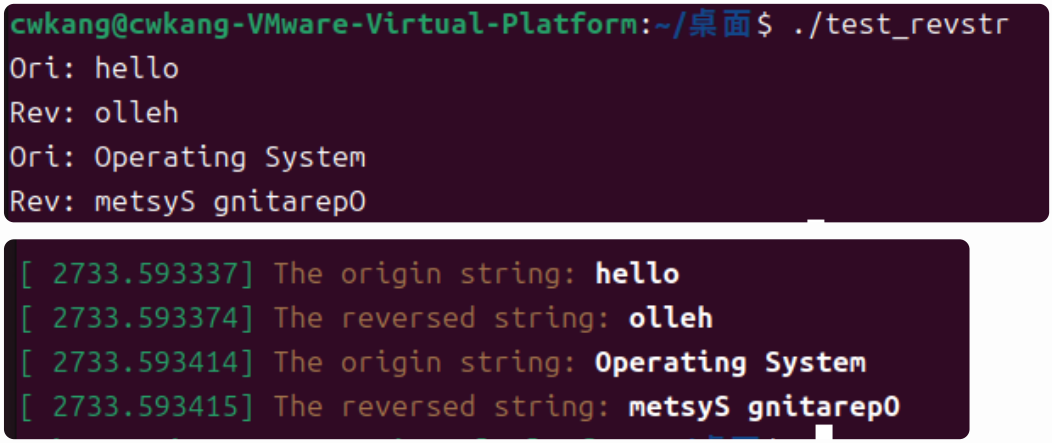
```
$ sudo update-grub
```

#### Step 9. Reboot the System

```
$ sudo reboot
```

## II. Implementing a new System Calls

### i. Answer screenshot




### ii. Describe the implementation details for the system call

#### Step 1. Create the System Call Source Code

Create the revstr directory and add the revstr.c file

```
$ mkdir revstr
$ vim revstr/revstr.c
```

In `revstr.c`, wirte the system call for reverse a string ,

- SYSCALL_DEFINE2: 2 means 2 parameters pass in

```c
#include <linux/kernel.h>
#include <linux/linkage.h>
#include <linux/syscalls.h>
#include <linux/uaccess.h>

SYSCALL_DEFINE2(revstr, char __user *, src, size_t, len) {
        char str[256] = {0};
        char rev_str[256] = {0};

        if(copy_from_user(str, src, len)){
                return -EFAULT;
        }

        str[len] = '\0';
        rev_str[len] = '\0';

        printk("The origin string: %s\n", str);

        for(int i = 0; i < len; i++){
                rev_str[len - i -1] = str[i];
        }
        printk("The reversed string: %s\n", rev_str);

        if(copy_to_user(src, rev_str, len+1)){
                return -EFAULT;
        }
        return 0;
}
```

### Step 2. Add the Module to the Makefile

Add a `Makefile` inside the revstr directory to define how to compile this module.

```
obj-y := hello_world.o
```

### Step 3. Update the Main Kernel Makefile

Edit the main `Makefile` in the root of the kernel source tree to include the revstr module.

`$ vim Makefile`

```
diff --git a/Makefile b/Makefile
index 7b1cf5e4a..fdb0407d5 100644
--- a/Makefile
+++ b/Makefile
@@ -743,7 +743,7 @@ endif

 ifeq ($(KBUILD_EXTMOD),)
 # Objects we will link into vmlinux / subdirs we need to visit
-core-y         :=
+core-y         := revstr/
 drivers-y      :=
 libs-y         := lib/
 endif # KBUILD_EXTMOD
```

#### 4. Modify the System Call Table

Update the system call table to register the new system call. In `syscall_64.tbl`, add a new entry to define the system call number and function name.

`$ vim arch/x86/entry/syscalls/syscall_64.tbl`

```
diff --git a/arch/x86/entry/syscalls/syscall_64.tbl b/arch/x86/entry/syscalls/syscall_64.tbl
index c84d12608..c1dc8239f 100644
--- a/arch/x86/entry/syscalls/syscall_64.tbl
+++ b/arch/x86/entry/syscalls/syscall_64.tbl
@@ -372,6 +372,7 @@
 448    common  process_mrelease        sys_process_mrelease
 449    common  futex_waitv             sys_futex_waitv
 450    common  set_mempolicy_home_node sys_set_mempolicy_home_node
+451    common  revstr                  sys_revstr
```

#### 5. Declare the System Call

In `syscalls.h`, declare the new system call so the kernel knows about it.

- marked as asmlinkage to match the way that system calls are invoked

`$ vim include/linux/syscalls.h`

```
diff --git a/include/linux/syscalls.h b/include/linux/syscalls.h
index a34b0f9a9..84a8415d0 100644
--- a/include/linux/syscalls.h
+++ b/include/linux/syscalls.h
@@ -1385,4 +1385,5 @@ int __sys_getsockopt(int fd, int level, int optname, char __user *optval,
                int __user *optlen);
 int __sys_setsockopt(int fd, int level, int optname, char __user *optval,
                int optlen);
+asmlinkage long sys_revstr(char __user* src, size_t len);
 #endif
```

#### 6. Compile the Kernel

Once all the changes are made, recompile the kernel and install the new modules.

```
$ make -j4
$ make modules -j4
$ sudo make modules_install
$ sudo make install
$ sudo update-grub
$ reboot
```

### 7. Test with TA's code

In `test_revstr.c`

```c
#include <unistd.h>
#include <string.h>
#include <stdio.h>
#include <assert.h>

#define __NR_revstr 451

int main(int argc, char *argv[]) {

    char str1[20] = "hello";
    printf("Ori: %s\n", str1);
    int ret1 = syscall(__NR_revstr, str1, strlen(str1));
    assert(ret1 == 0);
    printf("Rev: %s\n", str1);

    char str2[20] = "Operating System";
    printf("Ori: %s\n", str2);
    int ret2 = syscall(__NR_revstr, str2, strlen(str2));
    assert(ret2 == 0);
    printf("Rev: %s\n", str2);

    return 0;
}
```

Compile and run the program:

```
$ vim test_revstr.c
$ gcc test_revstr.c -o test_revstr
$ ./test_revstr
$ sudo dmesg
```